

UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

DEPARTMENT OF INTERNET OF THINGS AND ROBOTICS ENGINEERING (IRE)

FACULTY OF CYBER PHYSICAL SYSTEM ENGINEERING

LAB REPORT 01

DIGITAL WALLET BACKEND SYSTEM

(Encapsulation, Inheritance, and Polymorphism in Dart)

COURSE CODE: IRE 460

COURSE TITLE: MOBILE PLATFORM FOR IOT DEVICES SESSIONAL

SUBMITTED BY:

Shahriar Alom Masud

Student ID: **2201032** | Session: 2022–2023

Department of Internet of Things and Robotics Engineering (IRE), UFTB

SUBMITTED TO:

Md. Rafiqul Islam

Lecturer

Department of Internet of Things and Robotics Engineering (IRE)

University of Frontier Technology, Bangladesh

NOVEMBER 03, 2026

1. Problem Statement

You need to create a simple backend system for a Digital Wallet app that handles two types of accounts: **Standard Accounts** and **Premium Accounts**. Premium users earn cash-back rewards whenever they deposit money.

2. Architecture Requirements

---

1. **Encapsulation (`BankAccount` class):**
   - Create a base class `BankAccount`.
   - Keep the `_balance` field private.
   - Provide a public getter to read the balance, but do not allow direct modifications from outside the class.
   - Implement a `deposit(double amount)` method.
2. **Inheritance & Polymorphism (`PremiumAccount` class):**
   - Create a subclass `PremiumAccount` that extends `BankAccount`.
   - Override the `deposit(double amount)` method so that premium users get an extra **5% cash-back bonus** added to their deposit automatically.

3. Solution Design

- **Encapsulation:** In Dart, a leading underscore (_) marks a variable library-private. By defining `double _balance = 0.0;` and exposing only `double get balance => _balance;`, direct unauthorized modification is prohibited while permitting safe read operations.
- **Inheritance:** The `PremiumAccount` class extends `BankAccount`, inheriting the balance management functionality without code duplication.
- **Polymorphism & Method Overriding:** The `deposit` method is overridden in `PremiumAccount` with the `@override` annotation. The 5% reward bonus is computed as:

  $$\text{Bonus} = \text{amount} \times 0.05$$

  The total amount deposited into the account is:

  $$\text{Total Credited} = \text{amount} + \text{Bonus} = \text{amount} \times 1.05$$

  Using `super.deposit(amount + bonus);`, the parent method safely handles updating the encapsulated balance.

GitHub Repository Link:
https://github.com/Masud744/flutter-lab/tree/main/Lab_01

4. Complete Dart Source Code (digital_wallet.dart)

```dart
//
    ================================================================================

// Course Code: IRE 460 | Mobile Platform for IoT Devices Sessional
// Lab Task: Digital Wallet Backend System
// Student Name: Shahriar Alom Masud | ID: 2201032
//
    ================================================================================


// Base class demonstrating Encapsulation
class BankAccount {
  // Private field
  double _balance = 0.0;

  // Public getter to read balance
  double get balance => _balance;

  // Method to deposit money
  void deposit(double amount) {
    if (amount > 0) {
      _balance += amount;
      print('Deposited: \$${amount.toStringAsFixed(2)} | New Balance: \$${
          _balance.toStringAsFixed(2)}');
    } else {
      print('Invalid deposit amount.');
    }
  }
}

// Subclass demonstrating Inheritance and Polymorphism
class PremiumAccount extends BankAccount {
  // Overriding deposit method to add 5% cash-back bonus automatically
  @override
  void deposit(double amount) {
    if (amount > 0) {
      double bonus = amount * 0.05; // 5% cash-back
      print('Premium Reward: 5% Cash-Back Bonus of \$${bonus.toStringAsFixed
          (2)} added!');
      super.deposit(amount + bonus);
    } else {
      print('Invalid deposit amount.');
    }
  }
}

void main() {
  print('--- Testing Standard Account ---');
  BankAccount standard = BankAccount();
  standard.deposit(1000.0);
  print('Final Standard Account Balance: \$${standard.balance.toStringAsFixed
      (2)}\n');

  print('--- Testing Premium Account ---');
  PremiumAccount premium = PremiumAccount();
  premium.deposit(1000.0);
  print('Final Premium Account Balance: \$${premium.balance.toStringAsFixed(2)
      }');
}
```

5. Program Execution Output

Below is the terminal output obtained by executing the Dart script (`dart run digital_wallet.dart`):

```
--- Testing Standard Account ---
Deposited: $1000.00 | New Balance: $1000.00
Final Standard Account Balance: $1000.00

--- Testing Premium Account ---
Premium Reward: 5% Cash-Back Bonus of $50.00 added!
Deposited: $1050.00 | New Balance: $1050.00
Final Premium Account Balance: $1050.00
```

6. Result Analysis and Conclusion

- **Standard Account Verification:** Depositing $1000.00 increases the balance from $0.00 to $1000.00 without any additional bonus, as expected. The balance is accessible strictly via the getter method.
- **Premium Account Verification:** Depositing $1000.00 triggers the overridden `deposit` method, calculating a 5% cash-back bonus ($1000 \times 0.05 = \$50.00$). The final balance credited is $1050.00, confirming the automatic cash-back logic.
- **Conclusion:** All architectural requirements regarding encapsulation, inheritance, and polymorphic method overriding in Dart were successfully satisfied with clean and modular code.